

Tabla Resumen de Patrones de Diseño (GoF)

Categoría	Patrón	¿Para qué sirve? (Propósito Principal)
CREACIONALES (Se enfocan en el mecanismo de creación de objetos, aumentando la flexibilidad y reutilización del código existente.)	Singleton	Garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella.
	Factory Method	Define una interfaz para crear un objeto, pero deja que las subclases decidan qué clase instanciar. Permite delegar la creación.
	Abstract Factory	Proporciona una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.
	Builder	Separa la construcción de un objeto complejo de su representación, permitiendo que el mismo proceso de construcción cree diferentes representaciones.
	Prototype	Especifica los tipos de objetos a crear usando una instancia prototípica y crea nuevos objetos copiando este prototipo (clonación).
ESTRUCTURALES (Se ocupan de la composición de clases y objetos para formar estructuras más grandes,	Adapter	Convierte la interfaz de una clase en otra interfaz que el cliente espera. Permite que clases con

Categoría	Patrón	¿Para qué sirve? (Propósito Principal)
manteniendo la flexibilidad y eficiencia.)		interfaces incompatibles trabajen juntas.
	Bridge	Desacopla una abstracción de su implementación para que ambas puedan variar independientemente.
	Composite	Compone objetos en estructuras de árbol para representar jerarquías parte-todo. Permite tratar a objetos individuales y composiciones de manera uniforme.
	Decorator	Añade responsabilidades adicionales a un objeto dinámicamente. Proporciona una alternativa flexible a la herencia para extender funcionalidad.
	Facade	Proporciona una interfaz unificada para un conjunto de interfaces en un subsistema. Define una interfaz de nivel más alto que hace que el subsistema sea más fácil de usar.
	Flyweight	Usa compartición para soportar eficientemente grandes cantidades de objetos de grano fino. Reduce el uso de memoria mediante la reutilización.
	Proxy	Proporciona un sustituto o marcador de posición para otro objeto para controlar el acceso al

Categoría	Patrón	¿Para qué sirve? (Propósito Principal)
		mismo (ej. acceso remoto, control de costos, caché).
COMPORTAMIENTO (Se centran en los algoritmos y la asignación de responsabilidades entre objetos; no solo describen patrones de objetos o clases, sino también los patrones de comunicación entre ellos.)	Chain of Responsibility	Evita acoplar el remitente de una solicitud a su receptor dando a más de un objeto la oportunidad de manejar la solicitud. Encadena los objetos receptores.
	Command	Encapsula una solicitud como un objeto, permitiéndole parametrizar clientes con diferentes solicitudes, encolar o registrar solicitudes, y soportar operaciones deshacer.
	Interpreter	Dado un lenguaje, define una representación para su gramática junto con un intérprete que usa la representación para interpretar sentencias en el lenguaje.
	Iterator	Proporciona una manera de acceder secuencialmente a los elementos de un objeto agregado sin exponer su representación subyacente.
	Mediator	Define un objeto que encapsula cómo interactúan un conjunto de objetos. Promueve el acoplamiento débil al evitar que los objetos se refieran explícitamente entre sí.

Categoría	Patrón	¿Para qué sirve? (Propósito Principal)
	Memento	Captura y externaliza el estado interno de un objeto sin violar la encapsulación, de modo que el objeto pueda ser restaurado a este estado más tarde.
	Observer	Define una dependencia uno-a-muchos entre objetos, de modo que cuando un objeto cambia de estado, todos sus dependientes son notificados y actualizados automáticamente.
	State	Permite a un objeto alterar su comportamiento cuando su estado interno cambia. El objeto parecerá cambiar su clase.
	Strategy	Define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables. Permite que el algoritmo varíe independientemente de los clientes que lo usan.
	Template Method	Define el esqueleto de un algoritmo en una operación, difiriendo algunos pasos a las subclases. Permite redefinir ciertos pasos del algoritmo sin cambiar su estructura.
	Visitor	Representa una operación a realizar sobre los elementos de una estructura de objetos. Permite definir una nueva operación sin cambiar las clases

Categoría	Patrón	¿Para qué sirve? (Propósito Principal)
		de los elementos sobre los que opera.

Notas

1. **No son recetas mágicas:** El uso indebido de patrones puede generar sobre-ingeniería (*over-engineering*). Se deben aplicar solo cuando el problema de diseño lo justifica.
2. **Combinación:** En proyectos reales (como los que han evaluado en las partes II y III de sus parciales), los patrones rara vez aparecen aislados. Es común ver un **Singleton** gestionando la conexión a base de datos, usado dentro de un **Factory Method**, cuyos productos son observados por un **Observer**.
3. **Modernidad:** Aunque estos son los patrones clásicos (GoF, 1994), siguen siendo la base. Sin embargo, en lenguajes modernos (como Python, JavaScript/TypeScript, o Kotlin), algunos patrones se simplifican sintácticamente (ej. el *Observer* nativo en React o RxJS, o el *Singleton* mediante módulos en JS)